

Food Fiesta

Developer Onboarding Guide

Generated: 20 juillet 2026

Stack:

Java 21

Spring Boot 3.4.2

Thymeleaf

H2 / PostgreSQL

Spring Security

Maven

TABLE OF CONTENTS

- 1. Purpose
- 2. How to Start the Application
- 3. Application Architecture
- 4. Architecture Diagram
- 5. CI/CD & Deployment
- 6. Project Health & Technical Debt Tracking
- 7. Known Gotchas & Contribution Guide

1 — Purpose

Food Fiesta is a full-stack dining management web application built with Spring Boot 3.4.2 and Java 21. It provides a premium online food-ordering experience for customers while giving restaurant administrators a complete back-office console to manage the menu, users, orders, and staff accounts.

User Roles

Role	Access	How to authenticate
Admin	Full back-office: manage products, users, orders, other admins	Custom form login (<code>/login</code>) validated against the <code>admin</code> table; session-based
Customer	Browse menu, place orders, view order history	Custom form login (<code>/userLogin</code>) or Google OAuth2; session-based
Anonymous	Home, Products, About, Locate Us — read only	No authentication needed

Core Features

- Thymeleaf SSR UI with custom CSS and JavaScript

- Role-based dual login system (admin + customer) with optional Google OAuth2
- Product (food item) catalog with CRUD management from the admin panel
- Customer registration and self-service account management
- Order placement with automatic total calculation and order history
- Spring Data JPA persistence (H2 in-memory for local dev, PostgreSQL for production)
- Swagger / OpenAPI 3 documentation at `/swagger-ui/index.html`
- Health-check REST endpoint at `/api/health`

2 — How to Start the Application

Prerequisites

Tool	Version	Notes
JDK	21	Required. Set <code>JAVA_HOME</code> on Windows if needed.
Maven	optional	Maven Wrapper (<code>mvnw</code> / <code>mvnw.cmd</code>) is included — no separate install needed.
Docker + Docker Compose	any recent	Only required for the PostgreSQL Docker flow.

Quick Start (H2 — recommended for local dev)

Clone and run in one shot:

```
# Linux / macOS
cd watsonx-hive-minders/Food-Fiesta
./mvnw spring-boot:run

# Windows PowerShell
cd watsonx-hive-minders\Food-Fiesta
.\mvnw.cmd spring-boot:run

# If JAVA_HOME is not set on Windows, prepend:
$env:JAVA_HOME="C:\Program Files\Java\jdk-21.0.11"
.\mvnw.cmd spring-boot:run
```

Local URLs

URL	Purpose
<code>http://localhost:8080/</code>	Application home page
<code>http://localhost:8080/swagger-ui/index.html</code>	Swagger / OpenAPI interactive docs

URL	Purpose
<code>http://localhost:8080/h2-console</code>	H2 in-memory DB console
<code>http://localhost:8080/api/health</code>	Health-check endpoint (returns <code>"System is Online"</code>)

H2 Console Connection

JDBC URL: `jdbc:h2:mem:foodfiesta`
 User Name: `sa`
 Password: (leave blank)

⚠ H2 is in-memory — **all data is lost when the app stops**. The DataLoader re-seeds sample products and the default admin account on every startup.

Default Seeded Credentials

Role	Email	Password
Admin	<code>admin@foodfiesta.com</code>	<code>admin123</code>
Customer	<code>register at /register</code>	<code>chosen at registration</code>

Docker / PostgreSQL Setup

```
# Build image and start app + PostgreSQL together
docker compose up -d

# Follow application logs
docker compose logs -f app
```

The app will be available on `http://localhost:8080` . PostgreSQL runs on port `5432` .

Environment Variables

Variable	Default	Purpose
<code>SPRING_DATASOURCE_URL</code>	<code>jdbc:postgresql://db:5432/foodfiesta</code>	JDBC URL (Docker / prod only)
<code>SPRING_DATASOURCE_USERNAME</code>	<code>postgres</code>	DB username
<code>SPRING_DATASOURCE_PASSWORD</code>	<code>password</code>	DB password —

Variable	Default	Purpose
		change in production!
<code>spring.security.oauth2.client.registration.google.client-id</code>	<code>YOUR_CLIENT_ID</code>	Google OAuth2 (optional)
<code>spring.security.oauth2.client.registration.google.client-secret</code>	<code>YOUR_CLIENT_SECRET</code>	Google OAuth2 (optional)

3 — Application Architecture

Package Structure

Package	Role	Classes
<code>config</code>	Spring configuration beans	<code>SecurityConfig</code> , <code>DataLoader</code> , <code>OpenApiConfig</code>
<code>controllers</code>	Spring MVC controllers (Thymeleaf views + REST)	<code>HomeController</code> , <code>AdminController</code> , <code>UserController</code> , <code>ProductController</code> , <code>OrderController</code> , <code>OAuth2Controller</code> , <code>TestController</code>
<code>services</code>	Business logic (annotated <code>@Component</code> , not <code>@Service</code>)	<code>AdminServices</code> , <code>UserServices</code> , <code>ProductServices</code> , <code>OrderServices</code>
<code>repositories</code>	Spring Data JPA repositories	<code>AdminRepository</code> , <code>UserRepository</code> , <code>ProductRepository</code> , <code>OrderRepository</code>
<code>entities</code>	JPA entities (DB tables)	<code>User</code> , <code>Admin</code> , <code>Product</code> , <code>Orders</code>
<code>loginCredentials</code>	Form-binding DTOs	<code>AdminLogin</code> , <code>UserLogin</code>
<code>count</code>	Utility / pure logic	<code>Logic</code> (static order total calculator)

Authentication Mechanism

Food Fiesta uses a **hybrid authentication model**:

- **Custom session-based login for admins** — `/adminLogin` (POST) manually checks email + plaintext password against the `admin` table via `AdminServices.validateAdminCredentials()` . On success, the admin email is stored in `HttpSession` as `"loggedInAdmin"` . Spring Security is *not* involved in this flow.
- **Custom session-based login for customers** — `/userLogin` (POST) manually checks email + plaintext password via `UserServices.validateLoginCredentials()` . On success, the full `User` JPA entity is stored in `HttpSession` as `"loggedInUser"` .
- **Google OAuth2 for customers** — Spring Security's OAuth2 client handles the Google flow; on success `OAuth2Controller.oauth2Success()` creates or retrieves a local `User` and puts it in the session like a normal login.

- **Spring Security** is configured to permit a fixed list of public routes and require authentication for everything else. However, the admin and user dashboards rely on manual session checks inside controller methods, *not* on Spring Security's access rules.
- **CSRF is disabled** — intentional (simplifies Thymeleaf form handling), but is a known security gap.
- Passwords for customers are stored and compared in **plaintext** — BCryptPasswordEncoder is wired in the bean but never applied in registration or login flows.

Data Model

Entity class	DB table	Key fields	Relationships
User	customer_info	u_id , uname , uemail , upassword , unumber	OneToMany → Orders
Admin	admin	adminId , adminName , adminEmail , adminPassword , adminNumber	none
Product	product_table	pid , pname , pprice , pdescription	none
Orders	order_info	oId , oName , oPrice , oQuantity , orderDate , totalAmmout	ManyToOne → User (FK user_u_id)

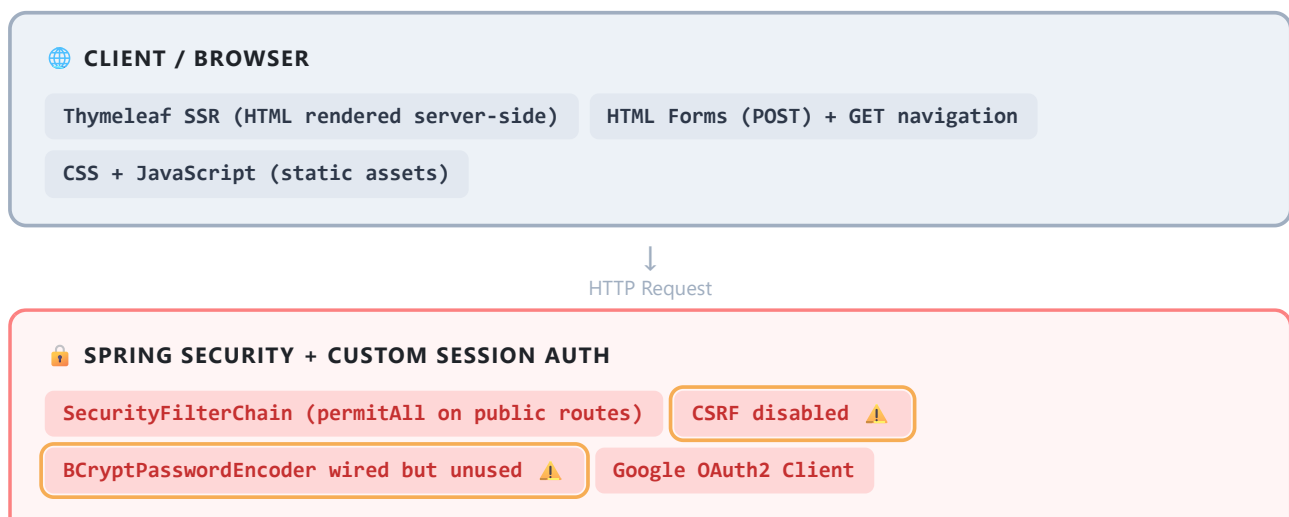
Request Lifecycle

Browser → Spring Security filter chain (permit public / block protected) → Controller method → Service method → Repository (Spring Data JPA / Hibernate) → H2 or PostgreSQL → entity returned → Thymeleaf template rendered → HTTP response sent to browser.

Seeded Data on Startup

DataLoader (a @Configuration + CommandLineRunner bean) seeds 5 products and 1 default admin account if the tables are empty. It runs on every startup since H2 uses create-drop .

4 — Architecture Diagram



H2 Console (sameOrigin frame policy)

↓
Authorised requests

🎮 CONTROLLERS (SPRING MVC @CONTROLLER / @RESTCONTROLLER)

HomeController (public pages)

AdminController ⚠️ God class (login + CRUD + orders)

UserController (user CRUD)

ProductController (product CRUD)

OAuth2Controller (Google login callback)

TestController (health check REST)

↓
Service calls

⚙️ SERVICES (ANNOTATED @COMPONENT — NOT @SERVICE)

AdminServices ⚠️ plaintext password comparison

UserServices ⚠️ plaintext password + full-table scan login

ProductServices

OrderServices

Logic (static utility – order total)

↓
Spring Data JPA / Hibernate

🗃️ REPOSITORIES (SPRING DATA JPA)

AdminRepository

UserRepository

ProductRepository

OrderRepository

↓
SQL (JDBC)

💾 DATABASE

H2 in-memory (local dev) – create-drop

PostgreSQL (Docker / production)

customer_info — User

admin — Admin

product_table — Product

order_info — Orders (FK user_u_id)

↕
Session / State

💡 HTTP SESSION

loggedInUser → User entity (full JPA object in session) ⚠️

loggedInAdmin → String (admin email)

5 — CI/CD & Deployment

Continuous Integration — GitHub Actions

File: `.github/workflows/trigger-onboarding-agent.yml` (root repo) and `watsonx-hive-minders/.github/workflows/trigger-onboarding-agent.yml`

Property	Value
Trigger	Push to <code>main</code> branch when source files change (<code>src/**</code> , <code>pom.xml</code> , <code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>application.properties</code>)
Manual trigger	Yes — <code>workflow_dispatch</code> with optional reason input
Runner	<code>ubuntu-latest</code>
What it does	Updates the generation date in <code>devOnboarding.html</code> and <code>architecture.md</code> , appends a change-log entry to <code>architecture.md</code> , then commits and pushes updated docs back to the repo
Commit message pattern	<code>docs: auto-update onboarding docs [ci skip] (<sha>)</code>
Permissions	<code>contents: write</code> (GITHUB_TOKEN)

Docker

The `Dockerfile` uses a **two-stage build**:

Stage	Base image	What it does
build	<code>eclipse-temurin:21-jdk-alpine</code>	Resolves dependencies offline, runs <code>./mvnw clean package -DskipTests</code> , produces <code>target/*.jar</code>
run	<code>eclipse-temurin:21-jre-alpine</code>	Copies only the JAR + <code>entrypoint.sh</code> , creates a non-root user <code>spring:spring</code> , exposes port 8080

Security measures: non-root user (`spring:spring`), minimal JRE-only runtime image.

docker-compose Services

Service	Image	Port	Notes
<code>app</code>	Built from local <code>Dockerfile</code>	8080→8080	Waits for <code>db</code> health check before starting
<code>db</code>	<code>postgres:latest</code>	5432→5432	Health check: <code>pg_isready -U postgres</code> every 10s (5s timeout, 5 retries)

AWS CodeBuild (buildspec.yml)

Phase	Commands
install	Logs Maven and Java versions
build	<code>mvn clean package -DskipTests</code>

Phase	Commands
post_build	Copies JAR to <code>output/</code>

Artifacts produced: `output/*.jar` , `appspec.yml` , `scripts/**/*`

Environment Variables — Production Checklist

Variable	Default	Must change in prod?
<code>SPRING_DATASOURCE_URL</code>	<code>jdbc:postgresql://db:5432/foodfiesta</code>	Yes — point to production DB host
<code>SPRING_DATASOURCE_USERNAME</code>	<code>postgres</code>	Yes
<code>SPRING_DATASOURCE_PASSWORD</code>	<code>password</code>	Yes — critical!
<code>spring.security.oauth2.client.registration.google.client-id</code>	<code>YOUR_CLIENT_ID</code>	Yes if Google OAuth2 is used
<code>spring.security.oauth2.client.registration.google.client-secret</code>	<code>YOUR_CLIENT_SECRET</code>	Yes if Google OAuth2 is used

Health Check

Endpoint: `GET /api/health` — returns the plain string `"System is Online"` (HTTP 200). Publicly accessible (no auth required).

6 — Project Health & Technical Debt Tracking

A **technical debt audit** is automatically run at the end of this onboarding session. Its full interactive report will be saved to `docs/audit/` and will give you an immediate view of the project's security, architecture, and quality health.

Re-running the Audit

To regenerate the full audit report at any time (with temporal delta showing improvement or regression vs. the previous baseline), open a new Bob conversation on this project and type:

```
lance un audit de dette technique
```


Baseline Tracking

The file `docs/audit/dette_technique_baseline.json` stores the reference snapshot. Every subsequent audit automatically compares against it and highlights what has improved or regressed.

Audit Deliverables

- `docs/audit/dette_technique.html` — interactive HTML report with KPI dashboard and prioritized refactoring backlog
- `docs/audit/dette_technique.pdf` — PDF version with debt visualization chart
- `docs/audit/dette_technique_baseline.json` — temporal tracking baseline

7 — Known Gotchas & Contribution Guide

Known Gotchas

⚠️ **Passwords are stored and compared in plaintext for customers.** `UserService.validateLoginCredentials()` does a plain `.equals(password)` comparison. `BCryptPasswordEncoder` is declared as a bean in `SecurityConfig` but is never injected into the registration or login flows. Do *not* "fix" this silently — it would break all existing user accounts in production PostgreSQL.

⚠️ **Admin passwords are also plaintext.** `AdminServices.validateAdminCredentials()` uses `.equals(password)`. The default seeded admin (`admin123`) is stored with no hashing.

⚠️ **Optional.get() without guard — potential NoSuchElementException.** `AdminServices.getAdmin(id)`, `UserService.getUser(id)`, and `ProductServices.getProduct(id)` all call `optional.get()` directly. If an ID that doesn't exist is passed (e.g. via a crafted URL), the application will throw an unguarded exception. The project catches this globally in `Exceptions.java`, but the UX is a generic error page.

⚠️ **All service classes use @Component instead of @Service.** `AdminServices`, `UserService`, `ProductServices`, and `OrderServices` are all annotated `@Component`. This works in Spring but goes against convention. Do *not* rename the annotation without a team discussion — it is intentional in this codebase.

⚠️ **Full JPA entity stored in HttpSession as loggedInUser.** The `User` entity (with its lazily loaded orders list) is stored directly in the HTTP session. This can cause `LazyInitializationException` or stale-data issues after the Hibernate session closes. Do not replace this with a DTO without testing all session-dependent flows.

⚠️ **Mutation endpoints mapped to @GetMapping.** `/updatingAdmin/{id}`, `/updatingUser/{id}`, `/updatingProduct/{productId}`, `/deleteAdmin/{id}`, `/deleteUser/{id}`, `/deleteProduct/{productId}` all use GET — not PUT/DELETE. This is how the Thymeleaf forms are wired. Do *not* change the HTTP verbs without updating all templates.

i **totalAmmout has a double letter — this is intentional.** The field is spelled `totalAmmout` (double *m*, missing *n*) in both the `Orders` entity and the `order_info` DB column. The getters/setters match. Never "fix" the spelling — it would require a

DB migration.

i CSRF is disabled project-wide. SecurityConfig calls `csrf(csrf -> csrf.disable())`. This is intentional to keep Thymeleaf form submissions simple. Do not re-enable without adding CSRF tokens to all forms.

i The main class is named FoodFrenzyApplication, not FoodFiestaApplication. The artifact name is "Food Fiesta" but the main class kept its original name. Don't rename it — it changes the JAR entry point.

Contribution Guide

Action	Command
Run the full test suite	<code>./mvnw test</code> (Linux/macOS) or <code>.\mvnw.cmd test</code> (Windows)
Build the JAR	<code>./mvnw clean package -DskipTests</code>
Run locally before pushing	<code>./mvnw spring-boot:run</code>
Run with Docker + PostgreSQL	<code>docker compose up -d</code>

- **All existing tests must pass** before pushing — the test suite covers controllers, services, repositories, and config.
- **Branching:** The CI workflow triggers on push to `main`. Use feature branches and open a PR to `main`; avoid direct pushes to `main` in shared environments.
- **CI checks that run on push:** The GitHub Actions onboarding workflow updates and commits docs automatically (tagged `[ci skip]` to avoid loops). AWS CodeBuild (`buildspec.yml`) runs `mvn clean package -DskipTests` and produces the deployment JAR.
-  **If you add a new service method, add a matching test in the corresponding `*Test.java` file** (e.g. add to `AdminServicesTest.java` for `AdminServices`).